

Ex.No.01	To retrieve Sensor data and practice conditional processing of the data using Raspberry Pi.
Date:	

AIM:

To interface a temperature sensor and an ultrasonic sensor with Raspberry Pi 3 and retrieve temperature, humidity, and distance measurements using Python programming.

SOFTWARE REQUIRED:

- Raspberry Pi OS
- Thonny Python IDE

HARDWARE REQUIRED:

S.No	Hardware
1.	Raspberry Pi 3 Model B
2.	Personal Computer
3.	HDMI Cable
4.	USB Keyboard
5.	Power Adapter

COMPONENTS REQUIRED:

S.NO	Component/Specification	Quantity
1.	Temperature/DHT21	1
2.	Ultrasonic Sensor/HC-SR04	1
3.	Breadboard	1
4.	Jumper Wires	As required

THEORY:

Raspberry Pi 3

The Raspberry Pi 3 Model B is a low-cost, credit-card-sized single-board computer widely used in IoT, embedded systems, robotics, and automation. It features a 1.2 GHz 64-bit quad-core ARM Cortex-A53 processor, 1 GB RAM, built-in Wi-Fi, Bluetooth, Ethernet, HDMI, USB ports, and a 40-pin General Purpose Input/Output (GPIO) header.

The GPIO pins allow the Raspberry Pi to communicate with various sensors and actuators by sending and receiving digital signals. Python is the most commonly used programming language for controlling GPIO pins due to its simplicity and extensive library support.

Temperature Sensor (DHT11)

The DHT11 is a digital sensor used to measure both temperature and relative humidity. It contains a thermistor for sensing temperature and a capacitive humidity sensor for measuring humidity. An internal microcontroller converts the measured values into digital data and transmits them to the Raspberry Pi through a single data pin.

The DHT11 operates at 3.3V to 5V, making it suitable for direct interfacing with the Raspberry Pi.

Specifications

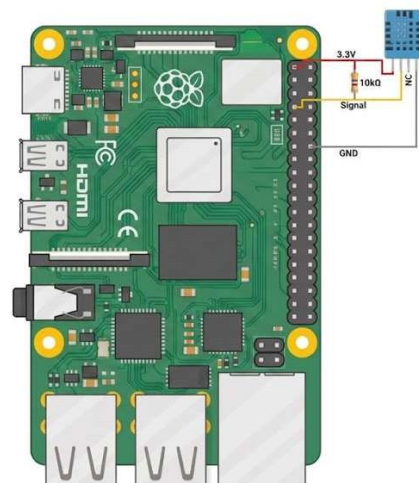
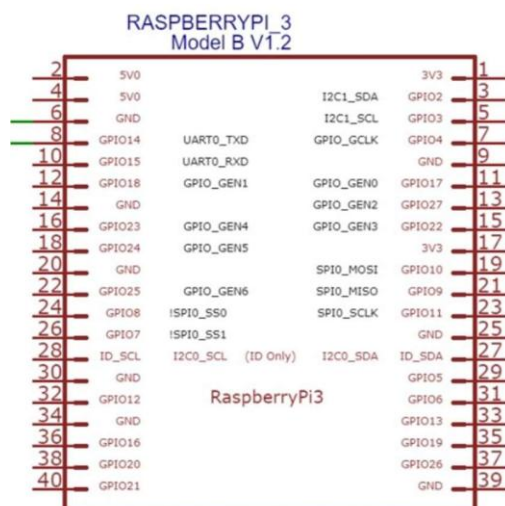
- Operating Voltage: 3.3V–5V
- Temperature Range: 0°C to 50°C
- Temperature Accuracy: $\pm 2^{\circ}\text{C}$
- Humidity Range: 20%–90% RH
- Humidity Accuracy: $\pm 5\%$ RH

Working Principle:

The DHT11 continuously senses the surrounding temperature and humidity. When requested by the Raspberry Pi, it sends the measured values digitally through the DATA pin. The Raspberry Pi processes these values using the Adafruit_DHT Python library and displays the results on the screen.

CIRCUIT DIAGRAM:

Temperature and Humidity Sensor Interfacing with Raspberry Pi



SOURCE CODE:

DHT11 Sensor

```
import Adafruit_DHT
import time
sensor = Adafruit_DHT.DHT11
pin = 4    # GPIO4
try:
    while True:
        humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)
        if humidity is not None and temperature is not None:
            print("Temperature = {} °C".format(temperature))
            print("Humidity = {} %".format(humidity))
            print("-----")
        else:
            print("Failed to retrieve data from sensor")
            time.sleep(2)
except KeyboardInterrupt:
    print("\nProgram Stopped")
```

PROCEDURE:

1. Connect the Raspberry Pi 3 to a monitor using an HDMI cable and connect the keyboard and mouse.
2. Power ON the Raspberry Pi and boot into Raspberry Pi OS.
3. Open the Thonny Python IDE or Terminal.
4. Connect the DHT11 temperature sensor:
 - VCC → 3.3V
 - GND → GND
 - DATA → GPIO4
5. Connect the HC-SR04 ultrasonic sensor:
6. VCC → 5V
 - GND → GND
 - TRIG → GPIO23
 - ECHO → GPIO24
7. Install the required Python libraries (such as Adafruit_DHT and RPi.GPIO) if they are not already installed.
8. Write the Python programs for interfacing the DHT11 and HC-SR04 sensors.

9. Save and execute the programs in the Thonny IDE.
10. Observe the temperature, humidity, and distance values displayed on the terminal
11. Verify that the displayed values change according to the surrounding environment and the distance of the object from the ultrasonic sensor.
12. Stop the program and safely shut down the Raspberry Pi after completing the experiment.

OUTPUT:

Shell ✕

```
Temperature: 33.99 °C
Temperature: 17.2 °C
Temperature: 96.01 °C
Temperature: 332.39 °C
Temperature: 314.25 °C
Temperature: 84.04 °C
Temperature: 313.74 °C
Temperature: 44.93 °C
Temperature: 38.49 °C
Temperature: 84.95 °C
Temperature: 35.87 °C
```

Sri Manakula Vinayagar Engineering College (An Autonomous Institution) Department of ECE	
ULC (5)	_____
ILS (5)	_____
TM (5)	_____
RAO (5)	_____
Viva (5)	_____
Total (25)	_____

RESULT:

The DHT11 temperature sensor and HC-SR04 ultrasonic sensor were successfully interfaced with the Raspberry Pi 3 using Python. The Raspberry Pi successfully acquired and displayed the temperature (°C), humidity (%RH), demonstrating successful sensor interfacing and data acquisition through the GPIO pins.

OUTCOME:

Ex.No.02	Familiarizing Raspberry Pi kit, Sensors, Actuators and practicing the initialization and interfacing of the kit with different components.
Date:	

AIM:

To familiarize with Raspberry Pi 3 by interfacing a ultrasonic sensor to practice initialization, detection of distance using Python programming.

SOFTWARE REQUIRED:

- Raspberry Pi OS
- Thonny Python IDE

HARDWARE REQUIRED:

S.No	Hardware
1.	Raspberry Pi 3 Model B
2.	Personal Computer
3.	HDMI Cable
4.	USB Keyboard
5.	USB Mouse
6.	Power Adapter

COMPONENTS REQUIRED:

S.NO	Component/Specification	Quantity
1.	PIR Motion Sensor (HC-SR501)	1
2.	Servo Motor (SG90)	1
3.	Breadboard	1
4.	Jumper Wires	As required

THEORY:

The Raspberry Pi is a single-board computer used for controlling and interfacing with various electronic components such as sensors and actuators. It provides GPIO (General Purpose Input/Output) pins that can be configured as input or output for communicating with external devices.

An ultrasonic sensor is a distance-measuring sensor that uses high-frequency sound waves to detect the distance of an object. The commonly used HC-SR04 ultrasonic sensor consists of two main parts: a Trigger pin and an Echo pin.

The Trigger pin is used to send a short ultrasonic pulse. When the pulse reaches an object, it is reflected back towards the sensor. The Echo pin receives the reflected signal and provides a pulse whose duration corresponds to the time taken by the ultrasonic wave to travel to the object and return.

The distance is calculated using the speed of sound:

$$\text{Distance} = \frac{\text{Time} \times \text{Speed of Sound}}{2}$$

The division by 2 is required because the ultrasonic wave travels from the sensor to the object and back to the sensor.

For example, the speed of sound in air is approximately 343 m/s at room temperature. In practical applications, the distance can be calculated in centimetres using:

$$\text{Distance(cm)} = \frac{\text{Time}(\mu\text{s}) \times 0.0343}{2}$$

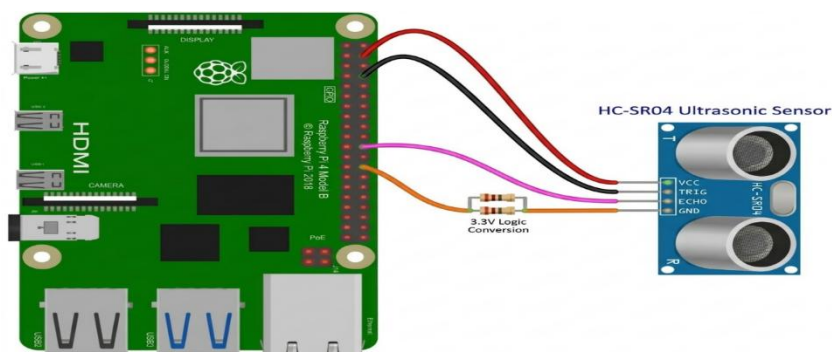
The Raspberry Pi reads the Echo pulse duration and calculates the distance. This value can then be displayed on the terminal or used to control an actuator such as an LED, buzzer, or motor.

Working Principle

1. Raspberry Pi sends a trigger signal to the Trigger pin of the HC-SR04.
2. The ultrasonic sensor transmits an ultrasonic pulse.
3. The pulse travels through the air and hits an object.
4. The reflected pulse returns to the sensor.
5. The Echo pin generates a signal corresponding to the travel time.
6. Raspberry Pi measures this time.
7. The distance is calculated using the ultrasonic distance equation.
8. The measured distance is displayed or used for further control.

CIRCUIT DIAGRAM:

Ultrasonic sensor interfacing with Raspberry Pi



PROCEDURE:

1. Connect the Raspberry Pi to a monitor, keyboard, and mouse.
2. Power ON the Raspberry Pi.
3. Open the Thonny Python IDE.
4. Connect the HC-SR04 Ultrasonic Sensor:
 - VCC → 5V
 - GND → GND
 - TRIG → GPIO19
 - ECHO → GPIO26
5. Connect the LED:
 - Anode (+) → GPIO5 through a suitable resistor
 - Cathode (−) → GND
6. Import the required Python libraries, RPi.GPIO and time.
7. Configure GPIO19 as the TRIG output, GPIO26 as the ECHO input, and GPIO5 as the LED output.
8. Write and save the Python program.
9. Execute the program.
10. Place an object in front of the ultrasonic sensor and observe the measured distance displayed in centimetres.
11. When the object is less than 20 cm from the sensor, observe that the LED turns ON.
12. Move the object beyond 20 cm and observe that the LED turns OFF.
13. Stop the program and safely shut down the Raspberry Pi.

SOURCE CODE:

Ultrasonic Sensor

```
import RPi.GPIO as GPIO
import time
TRIG = 19
ECHO = 26
LED = 5
GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.setup(LED, GPIO.OUT)
try:
    while True:
        GPIO.output(TRIG, 1)
        time.sleep(0.00001)
        GPIO.output(TRIG, 0)
        while not GPIO.input(ECHO):
            start = time.time()
        while GPIO.input(ECHO):
            end = time.time()
        distance = (end - start) * 17150
        print("Distance:", round(distance, 2), "cm")
        GPIO.output(LED, distance < 20)
        time.sleep(0.2)
except KeyboardInterrupt:
    GPIO.cleanup()
```

OUTPUT:

Shell ✕

```
Distance: 33.99 cm
Distance: 17.2 cm
Distance: 96.01 cm
Distance: 332.39 cm
Distance: 314.25 cm
Distance: 84.04 cm
Distance: 313.74 cm
Distance: 44.93 cm
Distance: 38.49 cm
Distance: 84.95 cm
Distance: 35.87 cm
```

Sri Manakula Vinayagar Engineering College (An Autonomous Institution) Department of ECE	
ULC (5)	_____
ILS (5)	_____
TM (5)	_____
RAO (5)	_____
Viva (5)	_____
Total (25)	_____

RESULT:

The experiment was conducted to interface the HC-SR04 ultrasonic sensor with Raspberry Pi for measuring the distance of an object and controlling an LED based on the measured distance. Thus, the experiment was successfully verified.

OUTCOME:

EX.NO: 03	To establish and understand communication protocols (MQTT and HTTP) to a IoT cloud server using Raspberry Pi.
Date:	

AIM

To establish and understand MQTT and HTTP communication protocols for sending sensor data from a Raspberry Pi to an IoT cloud server and to verify successful data transmission using both protocols.

COMPONENTS REQUIRED

1. Raspberry Pi 4
2. Wi-Fi Internet Connection

SOFTWARE REQUIRED

1. Raspberry Pi OS
2. Python 3
3. Paho-MQTT library
4. Requests library
5. Mosquitto MQTT Client
6. ThingSpeak Cloud Account

THEORY

The Internet of Things (IoT) connects physical devices such as sensors and controllers to the Internet so that data can be collected, transmitted, stored, and monitored remotely. In this experiment, a Raspberry Pi is used as the IoT device. The Raspberry Pi collects temperature and humidity values from a DHT11 sensor and sends the data to an IoT cloud server using two different communication protocols: MQTT and HTTP.

1. MQTT Protocol

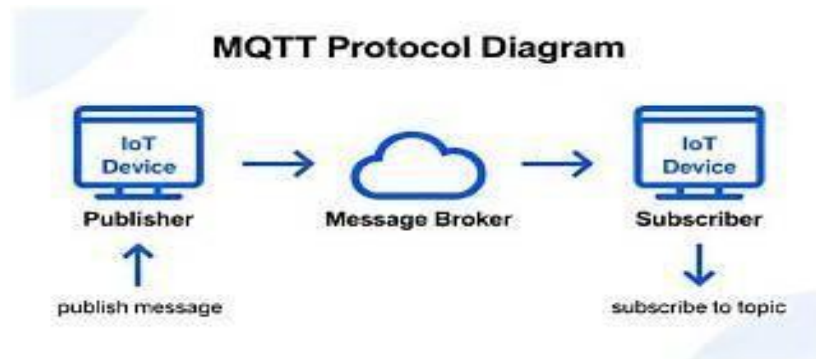
MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol designed especially for IoT applications. It is useful for devices having limited processing power, memory, bandwidth, and energy.

MQTT follows a Publish/Subscribe communication model. It mainly consists of:

- **Publisher** – The device that sends the data.
- **Broker** – The central server that receives and distributes messages.
- **Subscriber** – The device or application that receives the data.
- **Topic** – A named channel through which messages are exchanged.

In this experiment, the Raspberry Pi acts as the publisher. It reads temperature and humidity from the DHT11 sensor and publishes the readings to a particular MQTT topic. The MQTT broker receives the message and forwards it to subscribers subscribed to the same topic.

MQTT works over TCP and commonly uses port 1883 for unencrypted communication and 8883 for TLS-encrypted communication. It requires less bandwidth and is therefore suitable for continuous sensor-data transmission.

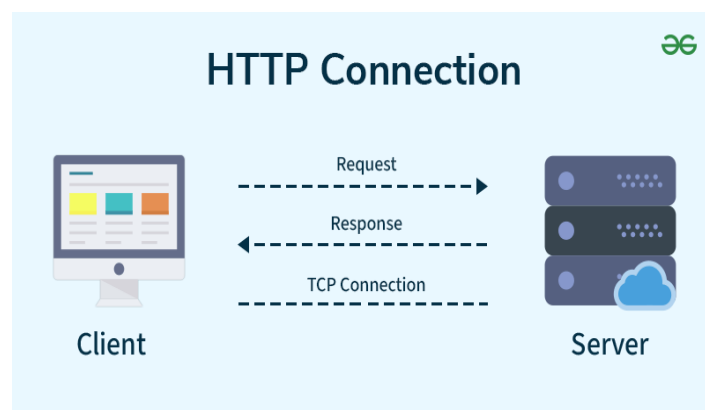


2. HTTP Protocol

HTTP (Hypertext Transfer Protocol) is another communication protocol used to transfer data between a client and a server. It follows a request/response model.

In this experiment, the Raspberry Pi acts as the client. It collects temperature and humidity values from the DHT11 sensor and sends them to the cloud server using an HTTP POST request. The cloud server processes the request and sends a response back to the Raspberry Pi.

HTTP is simple and widely supported. However, compared with MQTT, it has more communication overhead for each message. It is therefore suitable for periodic or occasional sensor-data uploads.



Installation Commands for MOTT:

```
sudo apt update
sudo apt upgrade
sudo apt install mosquitto mosquitto-clients -y
sudo systemctl start mosquitto
sudo systemctl enable mosquitto
sudo systemctl status mosquitto
```

MQTT Procedure:

STEP 1: Update Raspberry Pi

```
sudo apt update  
sudo apt upgrade -  
y
```

Wait until both commands finish.

STEP 2: Install Mosquitto MQTT Broker

```
sudo apt install mosquitto -y
```

This installs the MQTT broker.

STEP 3: Install Mosquitto MQTT Clients

```
sudo apt install mosquitto-clients -y
```

This installs mosquitto_pub (publisher) and mosquitto_sub (subscriber).

STEP 4: Start the Mosquitto Broker

```
sudo systemctl start mosquitto
```

Starts the MQTT broker.

STEP 5: Enable Mosquitto at Startup

```
sudo systemctl enable mosquitto
```

Makes Mosquitto start automatically when the Raspberry Pi boots.

STEP 6: Check Whether Mosquitto Is Running

```
sudo systemctl status mosquitto
```

Expected output should contain: Active: active (running). Press q to exit the status screen.

STEP 7: Check the MQTT

```
Commands mosquitto_pub --  
help
```

```
mosquitto_sub --help
```

Help information should be displayed for both commands.

STEP 8: Open Two Terminals

Use Terminal 1 as the subscriber and Terminal 2 as the publisher.

STEP 9: Start the MQTT Subscriber

```
mosquitto_sub -h localhost -t iot/sensor
```

Run this in Terminal 1. Leave it running; it waits for MQTT messages.

STEP 10: Publish a Message

```
mosquitto_pub -h localhost -t iot/sensor -m "Hello Raspberry
```

Pi" Run this in Terminal 2.

STEP 11: Verify the Message

Terminal 1 should display: "Hello Raspberry Pi". This confirms MQTT communication.

STEP 12: Send Sensor Data

```
mosquitto_pub -h localhost -t iot/sensor -m "Temperature: 28
C" mosquitto_pub -h localhost -t iot/sensor -m "Humidity:
65%" mosquitto_pub -h localhost -t iot/sensor -m "Light: 450
lux"
```

Terminal 1 should display all three messages.

STEP 13: Continuous MQTT Communication

```
while true
do
mosquitto_pub -h localhost -t iot/sensor -m "Temperature: 28
C" sleep 2
done
```

Run in Terminal 2 to publish a message every 2 seconds. Press Ctrl+C to stop.

STEP 14: Stop the Subscriber

```
mosquitto_sub -h localhost -t iot/sensor
```

For a running subscriber, press Ctrl+C in Terminal 1 to stop it.

Output for MQTT Communication:

```
Temperature: 28 C
Humidity: 65%
MQTT communication successful
```

HTTP Procedure:

STEP 1: Check Internet Connection

```
ping -c 4 google.com
```

Expected: 4 packets transmitted, 4 received, 0% packet loss.

STEP 2: Update Package Information

```
sudo apt update
```

STEP 3: Install curl

```
sudo apt install curl -y
```

STEP 4: Verify curl

```
curl --version
```

A curl version should be displayed. HTTP and HTTPS should appear among the supported protocols.

STEP 5: Create a ThingSpeak Channel

On a laptop/phone, open ThingSpeak, sign in, then select Channels → My Channels → New Channel. Use channel name Raspberry Pi HTTP. Enable Field 1 = Temperature and Field 2 = Humidity, then save.

STEP 6: Get the Channel ID

Open the channel and note its numeric Channel ID.

STEP 7: Get the Write API Key

Open the channel's API Keys section and copy the Write API Key. Keep it private.

STEP 8: Send the First HTTP Request

Replace YOUR_WRITE_API_KEY with your actual key. The URL must be preceded by curl. curl

```
"https://api.thingspeak.com/update?api_key=YOUR_WRITE_API_KEY&field1=25"
```

This sends 25 to Field 1.

STEP 9: Check the Response

A successful request normally returns a positive ThingSpeak entry number, for example:

1

STEP 10: Verify ThingSpeak

Open the ThingSpeak channel and check that Field 1 contains the value 25.

STEP 11: Send Temperature and Humidity

```
curl
```

```
"https://api.thingspeak.com/update?api_key=YOUR_WRITE_API_KEY&field1=28&field2=65"
```

STEP 12: Send More Readings

```
curl
```

```
"https://api.thingspeak.com/update?api_key=YOUR_WRITE_API_KEY&field1=30&field2=70"
```

curl

"https://api.thingspeak.com/update?api_key=YOUR_WRITE_API_KEY&field1=32&field2=72" Allow about 15 seconds between updates on a standard free ThingSpeak channel.

OUTPUT:



Sri Manakula Vinayagar Engineering College (An Autonomous Institution) Department of ECE	
ULC (5)	_____
ILS (5)	_____
TM (5)	_____
RAO (5)	_____
Viva (5)	_____
Total (25)	_____

RESULT:

The given experiment is MQTT and HTTP communication protocols were successfully implemented using Raspberry Pi data was successfully transmitted to the IoT cloud server.

OUTCOME:



Ex.no:04	To analyse Sensor data from Raspberry Pi on a cloud server (AWS IoT Core or Azure IoT Hub) and visualize the analysis.
Date:	

AIM:

To interface an HC-SR04 ultrasonic sensor with Raspberry Pi, transmit the measured distance to AWS IoT Core using MQTT, verify the received data, analyze the measurements, and visualize the results.

HARDWARE REQUIRED:

1.	Raspberry Pi
2.	HC-SR04 Ultrasonic Distance Senso
3.	Resistor

SOFTWARE REQUIRED:

1.	Raspberry Pi OS
2.	Python 3
3.	AWS IoT Core
4.	AWS IoT Device SDK for Python v2
5.	Matplotlib
6.	Pandas
7.	MQTT

THEORY:

The HC-SR04 ultrasonic distance sensor is used to measure the distance between the sensor and an object by using ultrasonic sound waves. The sensor sends an ultrasonic pulse through the TRIG pin and waits for the reflected signal through the ECHO pin. The time taken for the ultrasonic wave to travel to the object and return to the sensor is measured by the Raspberry Pi.

The distance is calculated using the following formula:

$$\text{Distance} = (\text{Time} \times \text{Speed of Sound}) / 2$$

The speed of sound is approximately 34300 cm/s. The division by 2 is necessary because the measured time represents the complete round-trip journey of the ultrasonic wave—from the sensor to the object and back to the sensor.

The Raspberry Pi acts as the IoT device or gateway. It interfaces with the HC-SR04 sensor through its GPIO pins. In this experiment, BCM GPIO 4 is used as the TRIG pin and BCM GPIO 17 is used as the ECHO pin. Since the HC-SR04 ECHO signal can be 5 V while

Raspberry Pi GPIO operates at 3.3 V logic, a 1 k Ω /2 k Ω voltage divider is used to reduce the ECHO voltage to a suitable level.

The sensor is controlled using Python programming and the RPi.GPIO library. The Raspberry Pi generates a short trigger pulse and measures the duration of the echo pulse. The calculated distance is then converted into a JSON data format containing the sensor name, measured distance, and timestamp.

For cloud communication, the experiment uses AWS IoT Core and the MQTT protocol. MQTT is a lightweight publish/subscribe messaging protocol commonly used for IoT communication. The Raspberry Pi acts as the MQTT publisher and sends the sensor readings to a specified MQTT topic. The AWS IoT MQTT Test Client subscribes to the same topic and displays the received messages.

The sensor readings are transmitted in JSON format, which provides a simple structured representation of the sensor name, distance value, and timestamp. The experiment uses the MQTT topic `sdk/test/python` for publishing the readings.

AWS IoT Core provides secure communication between the Raspberry Pi and the cloud. The device uses an AWS IoT certificate, private key, and root CA certificate for authentication and secure connection. The MQTT connection uses QoS 1, which provides at-least-once message delivery.



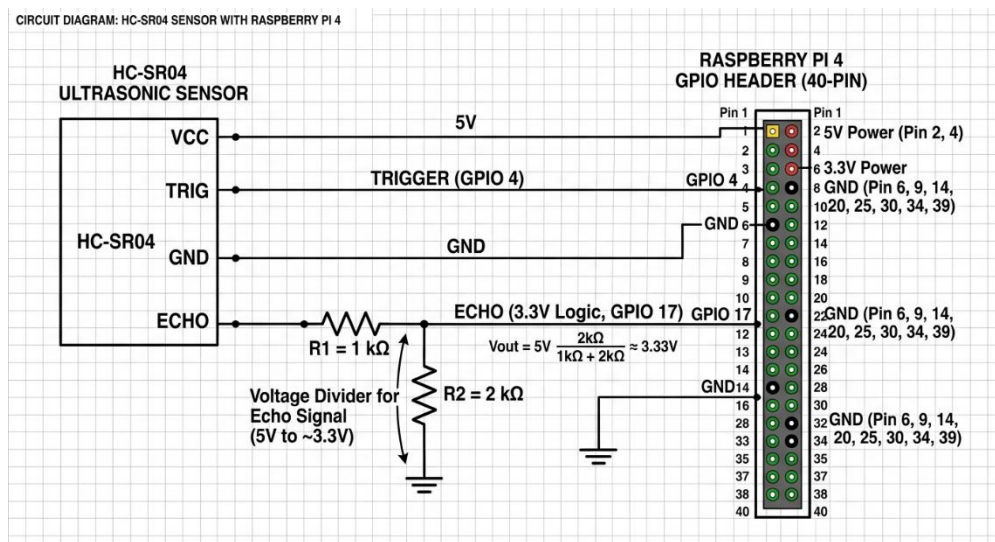
The collected sensor readings can also be stored in a CSV file for further analysis. Python libraries such as Pandas and Matplotlib are used to process and visualize the data. Statistical parameters such as minimum, maximum, average, median, and standard deviation are calculated to understand the variation in the measured distance.

Finally, the analyzed data is represented using a distance-versus-sample graph. The graph shows how the measured distance changes over successive sensor readings. Thus, the experiment demonstrates the complete IoT data flow from sensor → Raspberry Pi → MQTT → AWS IoT Core → data analysis → visualization.

PROCEDURE:

1. Connect HC-SR04 to Raspberry Pi.
2. Connect the ECHO pin through a 1 kΩ/2 kΩ voltage divider.
3. Configure the AWS IoT Core Thing and obtain the connection kit.
4. Create and activate a Python virtual environment.
5. Install the required GPIO and AWS SDK packages.
6. Test the HC-SR04 sensor and verify distance readings.
7. Connect Raspberry Pi to AWS IoT Core using MQTT.
8. Publish sensor readings as JSON data.
9. Subscribe to the MQTT topic using AWS IoT MQTT Test Client.
10. Collect the sensor readings and save them in a CSV file.
11. Calculate minimum, maximum, average, median and standard deviation.
12. Plot the distance versus sample graph.

CIRCUIT DIAGRAM:



PROGRAM:

```
import RPi.GPIO as GPIO

import time

import json

from awscrt import mqtt

from awsiot import mqtt_connection_builder

TRIG = 4

ECHO = 17

GPIO.setmode(GPIO.BCM)

GPIO.setup(TRIG, GPIO.OUT)

GPIO.setup(ECHO, GPIO.IN)

GPIO.output(TRIG, GPIO.LOW)

ENDPOINT = "a2p3dz68mtm692-ats.iot.ap-south-1.amazonaws.com"

CLIENT_ID = "basicPubSub"

CERTIFICATE = "./RaspberryPi_Exp4.cert.pem"

PRIVATE_KEY = "./RaspberryPi_Exp4.private.key"

ROOT_CA = "./root-CA.crt"

TOPIC = "sdk/test/python"

mqtt_connection = mqtt_connection_builder.mtls_from_path(

    endpoint=ENDPOINT,

    cert_filepath=CERTIFICATE,

    pri_key_filepath=PRIVATE_KEY,

    ca_filepath=ROOT_CA,

    client_id=CLIENT_ID,
```

```
        clean_session=True,

        keep_alive_secs=30
    )

    print("Connecting to AWS IoT...")

    mqtt_connection.connect().result()

    print("Connected to AWS IoT!")

    print("Publishing ultrasonic sensor data...")

    def get_distance():

        GPIO.output(TRIG, GPIO.LOW)

        time.sleep(0.05)

        GPIO.output(TRIG, GPIO.HIGH)

        time.sleep(0.00001)

        GPIO.output(TRIG, GPIO.LOW)

        timeout = time.time() + 0.1

        while GPIO.input(ECHO) == GPIO.LOW:

            if time.time() > timeout:

                return None

            pulse_start = time.time()

            timeout = time.time() + 0.1

            while GPIO.input(ECHO) == GPIO.HIGH:

                if time.time() > timeout:

                    return None

            pulse_end = time.time()

        return ((pulse_end - pulse_start) * 34300) / 2
```

```
try:

    while True:

        distance = get_distance()

        if distance is not None:

            distance = round(distance, 2)

            data = {

                "sensor": "HC-SR04",

                "distance_cm": distance,

                "timestamp": time.time()

            }

            payload = json.dumps(data)

            print("Distance:", distance, "cm")

            print("Publishing:", payload)

            mqtt_connection.publish(

                topic=TOPIC,

                payload=payload,

                qos=mqtt.QoS.AT_LEAST_ONCE

            )

        else:

            print("Ultrasonic sensor timeout")

            time.sleep(2)

    except KeyboardInterrupt:

        print("\nStopping...")

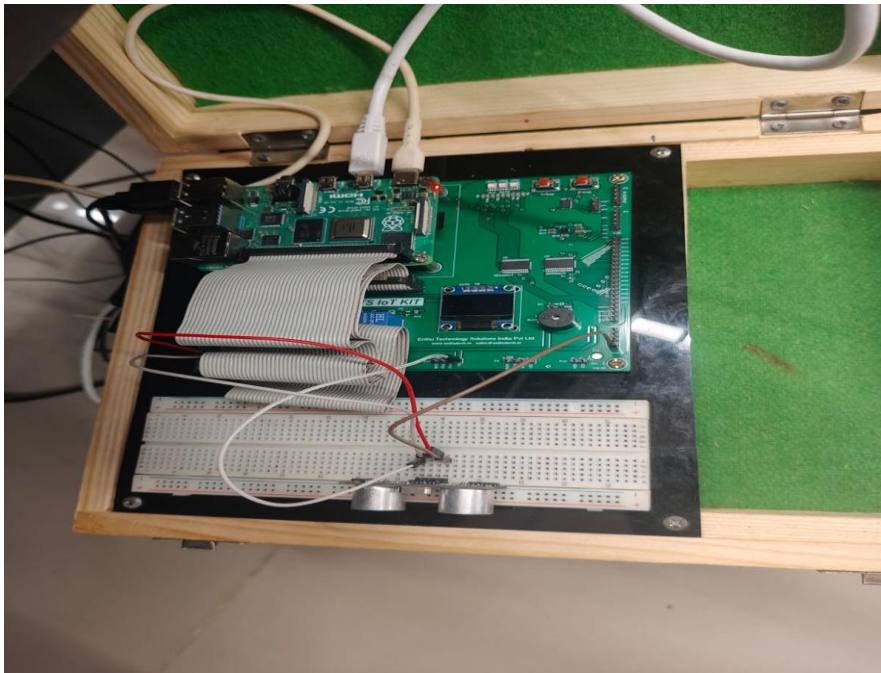
    finally:
```

```
GPIO.cleanup()

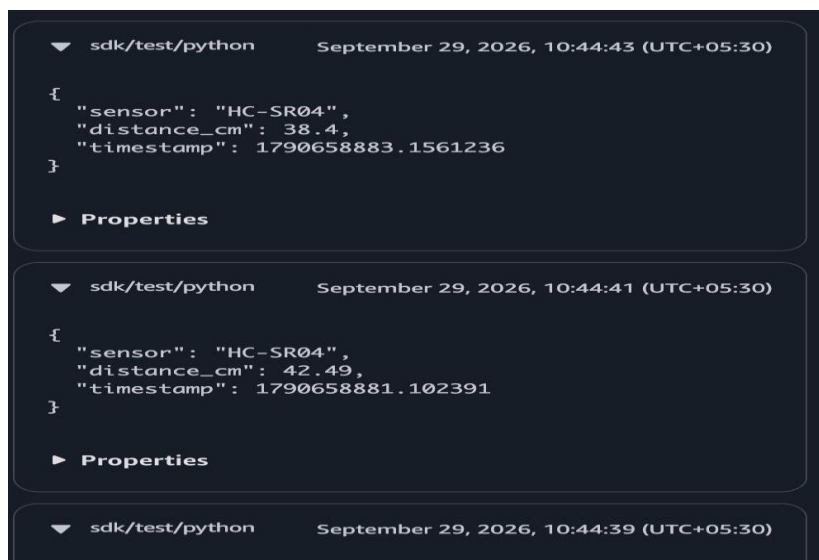
print("Disconnecting from AWS IoT...")

mqtt_connection.disconnect().result()

print("Disconnected.")
```



OUTPUT:



**Sri Manakula Vinayagar Engineering
College**

(An Autonomous Institution)

Department of ECE

ULC (5)	_____
ILS (5)	_____
TM (5)	_____
RAO (5)	_____
Viva (5)	_____
Total (25)	_____

RESULT:

The HC-SR04 ultrasonic sensor successfully measured distance and transmitted the readings from Raspberry Pi to AWS IoT Core using MQTT. The received data was verified using the AWS IoT MQTT Test Client, statistically analyzed, and visualized as a distance-versus-sample graph

OUTCOME:

Ex.no:05	To utilize Data visualization tools (ThingsBoard, Grafana, or Google Sheets) and present a real-time graph of the data acquired based on sensor thresholds the analysis.
Date:	

AIM

To measure object distance using an HC-SR04 ultrasonic sensor, interface it with a Raspberry Pi, transmit the measured distance to ThingsBoard Cloud using MQTT, visualize the distance in real time, and generate a threshold-based alarm.

HARDWARE REQUIRED:

1.	Raspberry Pi
2.	HC-SR04 Ultrasonic Distance Senso
3.	Resistor

SOFTWARE REQUIRED:

1.	Raspberry Pi OS
2.	Python 3
3.	RPi.GPIO
4.	Paho MQTT
5.	ThingsBoard Cloud
6.	MQTT protocol

THEORY

HC-SR04 Ultrasonic Sensor

The HC-SR04 is an ultrasonic distance measurement sensor used to determine the distance between the sensor and an object. It works by transmitting an ultrasonic sound pulse and measuring the time taken for the reflected pulse to return.

The sensor mainly uses two important pins:

- **TRIG** – used to trigger the ultrasonic pulse.
- **ECHO** – produces a pulse whose duration represents the time taken by the ultrasonic wave to travel to the object and return.

The Raspberry Pi sends a short trigger pulse to the TRIG pin. The HC-SR04 then transmits an ultrasonic signal. When this signal strikes an object, it gets reflected back toward the sensor. The ECHO pin remains HIGH for the duration corresponding to the round-trip travel time.

Distance Calculation

The distance is calculated using:

$$\text{Distance} = (\text{Time} \times \text{Speed of Sound}) / 2$$

The speed of sound is approximately:

34300 cm/s

Therefore,

Distance (cm) = Time (s) × 34300 / 2

The division by 2 is required because the measured time represents the total round-trip distance: from the sensor to the object and back to the sensor.

Raspberry Pi

The Raspberry Pi acts as the processing and communication unit. It receives the ECHO signal from the HC-SR04, calculates the distance using Python, and sends the measured value to ThingsBoard Cloud.

In this experiment:

- TRIG → BCM GPIO 4
- ECHO → BCM GPIO 17

The ECHO connection must use a voltage divider because the HC-SR04 ECHO signal can be approximately 5 V, whereas Raspberry Pi GPIO operates at 3.3 V logic.

MQTT

MQTT (Message Queuing Telemetry Transport) is a lightweight communication protocol commonly used in IoT systems.

Here, the Raspberry Pi works as the MQTT client and sends the measured distance to ThingsBoard Cloud.

ThingsBoard Cloud

ThingsBoard Cloud is used to receive, store, visualize, and process IoT telemetry. The Raspberry Pi publishes the distance value, and ThingsBoard displays it as telemetry.

A device named RaspberryPi_HCSR04 is created in ThingsBoard, and its access token is used for MQTT authentication.

Real-Time Monitoring

The measured distance is transmitted periodically to ThingsBoard using MQTT. A time-series chart can then be configured to display the variation of distance with time.

The telemetry key used for the graph is:

distance_cm

The experiment specifies a demonstration graph range of 0 to 70 cm when manual Y-axis limits are available.

Threshold Alarm

A threshold-based alarm is configured using the measured distance.

Alarm condition:

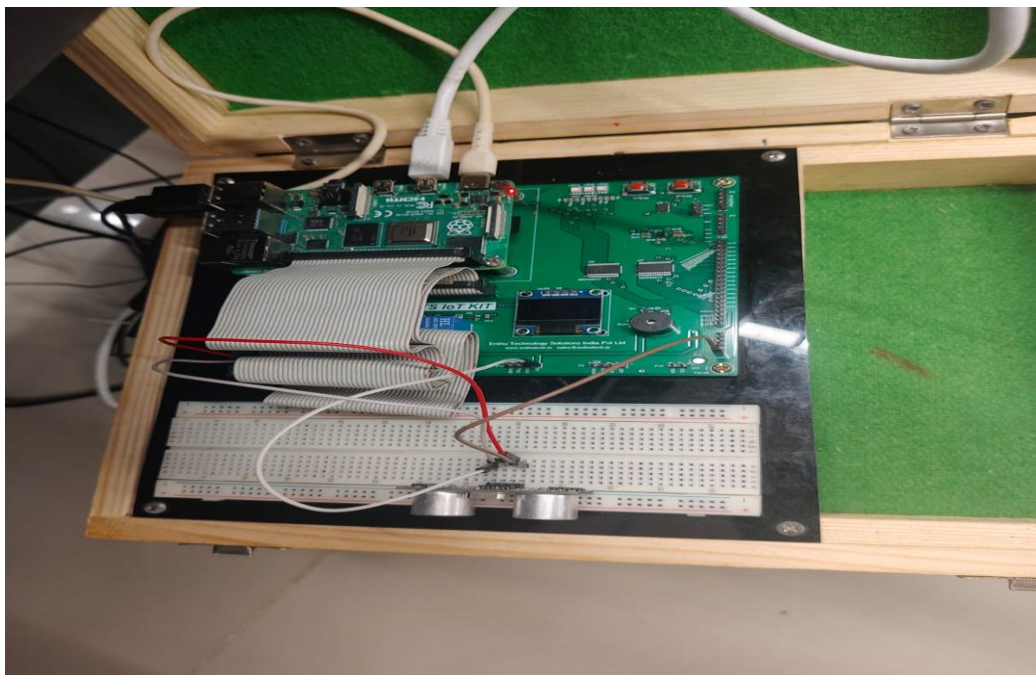
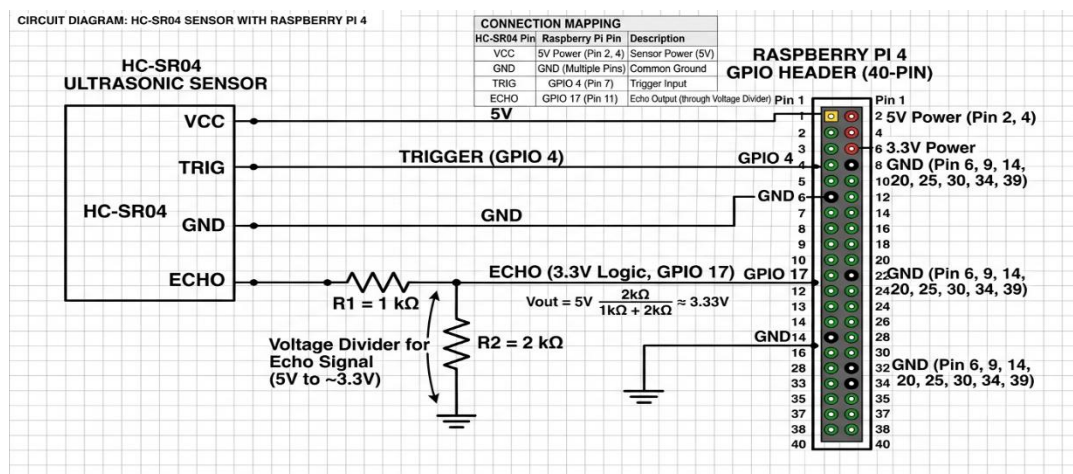
$\text{distance_cm} > 50 \text{ cm}$

Clear condition:

$\text{distance_cm} \leq 50 \text{ cm}$

Therefore, a measured distance above 50 cm produces a High Distance alarm, while a distance of 50 cm or below clears the alarm.

CIRCUIT DIAGRAM:



PROCEDURE USING THINGSBOARD:

Step 1: Connect the Hardware

1. Connect the HC-SR04 VCC to Raspberry Pi 5V.
2. Connect GND to GND.
3. Connect TRIG to BCM GPIO 4.
4. Connect ECHO to BCM GPIO 17 through the voltage divider.
5. Connect the Raspberry Pi to the Internet.

Step 2: Create Python Environment

Open the Raspberry Pi terminal and execute:

```
cd ~  
  
python3 -m venv exp5-env  
  
source exp5-env/bin/activate
```

Install the required libraries:

```
pip install RPi.GPIO  
  
pip install paho-mqtt
```

Verify the installations:

```
python3 -c "import RPi.GPIO; print('GPIO OK')"  
  
python3 -c "import paho.mqtt.client; print('Paho MQTT OK')"
```

Step 3: Create ThingsBoard Device

1. Log in to ThingsBoard Cloud.
2. Go to **Entities** → **Devices**.
3. Create a device named **RaspberryPi_HCSR04**.
4. Open the device.
5. Select **Manage credentials**.
6. Obtain the device Access Token.
7. Use the token in the Python program.

The access token should be kept private.

Step 4: Create Python Program

Create the file:

```
nano exp5_thingsboard_ultrasonic.py
```

Step 5: Enter the Program

```
import RPi.GPIO as GPIO
import paho.mqtt.client as mqtt
import json
import time

TRIG = 4
ECHO = 17

THINGSBOARD_HOST = "mqtt.thingsboard.cloud"
THINGSBOARD_PORT = 1883
ACCESS_TOKEN = "YOUR_ACCESS_TOKEN"
TOPIC = "v1/devices/me/telemetry"

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.output(TRIG, GPIO.LOW)

time.sleep(1)

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("MQTT connection successful.")
    else:
        print("MQTT connection failed. Code:", rc)

def on_disconnect(client, userdata, rc):
    print("MQTT disconnected. Code:", rc)

client = mqtt.Client()
client.username_pw_set(ACCESS_TOKEN)
client.on_connect = on_connect
```

```

client.on_disconnect = on_disconnect

def get_distance():
    GPIO.output(TRIG, GPIO.LOW)
    time.sleep(0.05)
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.00001)
    GPIO.output(TRIG, GPIO.LOW)
    start_wait = time.time()
    while GPIO.input(ECHO) == GPIO.LOW:
        if time.time() - start_wait > 0.1:
            return None
    pulse_start = time.time()
    while GPIO.input(ECHO) == GPIO.HIGH:
        if time.time() - pulse_start > 0.1:
            return None
    pulse_end = time.time()
    pulse_duration = pulse_end - pulse_start
    distance = (pulse_duration * 34300) / 2
    return round(distance, 2)

print()
print("=====")
print(" EXPERIMENT 5")
print(" HC-SR04 + Raspberry Pi + ThingsBoard")
print("=====")
print()
print("Connecting to ThingsBoard...")
try:
    client.connect(THINGSBOARD_HOST, THINGSBOARD_PORT, 60)

```

```
except Exception as error:
    print("Connection error:")
    print(error)
    GPIO.cleanup()
    exit()
client.loop_start()
time.sleep(2)
if client.is_connected():
    print("Connected to ThingsBoard!")
else:
    print("ERROR: MQTT connection was not established.")
    GPIO.cleanup()
    client.loop_stop()
    exit()
print()
print("Starting HC-SR04 telemetry...")
print("Press Ctrl+C to stop.")
print()
try:
    while True:
        print("Measuring distance...")
        distance = get_distance()
        if distance is None:
            print("Ultrasonic sensor timeout")
            print()
            time.sleep(2)
            continue
        print(f"Distance: {distance:.2f} cm")
```

```
data = {"distance_cm": distance}
payload = json.dumps(data)
result = client.publish(TOPIC, payload, qos=1)
if result.rc == mqtt.MQTT_ERR_SUCCESS:
    print(f"Sent to ThingsBoard: {payload}")
else:
    print("MQTT publish failed.")
    print("Error code:", result.rc)
print()
time.sleep(2)
except KeyboardInterrupt:
    print()
    print("Stopping Experiment 5...")
finally:
    GPIO.cleanup()
    client.loop_stop()
    client.disconnect()
    print("GPIO cleaned up.")
    print("Disconnected from ThingsBoard.")
    print("Experiment 5 stopped.")
```

step 7. run the program

Activate the environment:

```
source exp5-env/bin/activate
```

Run the Python program:

```
python3 exp5_thingsboard_ultrasonic.py
```

PROGRAM USING GOOGLESHEET:

```
Thonney id

import RPi.GPIO as GPIO

import time

import requests

# GPIO pins - PHYSICAL PIN NUMBERS

TRIG = 13

ECHO = 11

# Google Apps Script URL

SCRIPT_URL="https://script.google.com/macros/s/AKfycbyfS271_NvEws3Nb
gzOKbRqKVX1eOjE2Ghdc4-aw6sr0dqDBdoLe6uLrqxmpII5BljKmA/exec"

# Threshold

THRESHOLD = 20

GPIO.setmode(GPIO.BOARD)

GPIO.setup(TRIG, GPIO.OUT)

GPIO.setup(ECHO, GPIO.IN)

try:

    while True:

        print("Distance measurement in progress")

        # Trigger LOW

        GPIO.output(TRIG, False)

        time.sleep(2)

        # Trigger HIGH for 10 microseconds

        GPIO.output(TRIG, True)

        time.sleep(0.00001)

        GPIO.output(TRIG, False)

        # Wait for ECHO HIGH

        while GPIO.input(ECHO) == 0:

            pulse_start = time.time()
```

```

# Wait for ECHO LOW
while GPIO.input(ECHO) == 1:
    pulse_end = time.time()
# Calculate distance
pulse_duration = pulse_end - pulse_start
distance = pulse_duration * 17150
distance = round(distance, 2)
print("Distance:", distance, "cm")
# Check threshold
if distance < THRESHOLD:
    status = "OBJECT TOO CLOSE"
else:
    status = "NORMAL"
# Send to Google Sheets
try:
    response = requests.get(
        SCRIPT_URL,
        params={"distance": distance},
        timeout=10
    )
    print("Cloud:", response.text)
except requests.RequestException:
    print("Cloud: Transmission error")
print("-----")
time.sleep(2)
except KeyboardInterrupt:
    print("Program stopped")

```

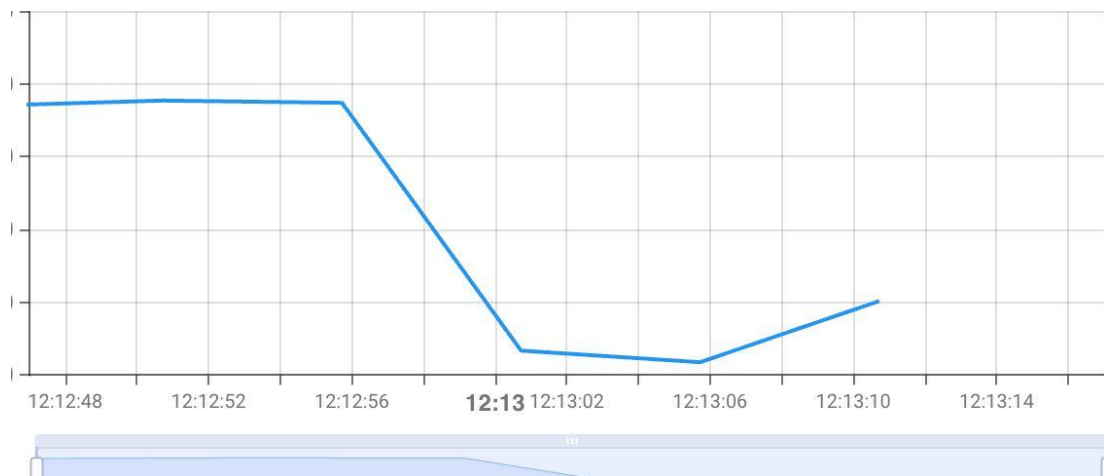
finally:

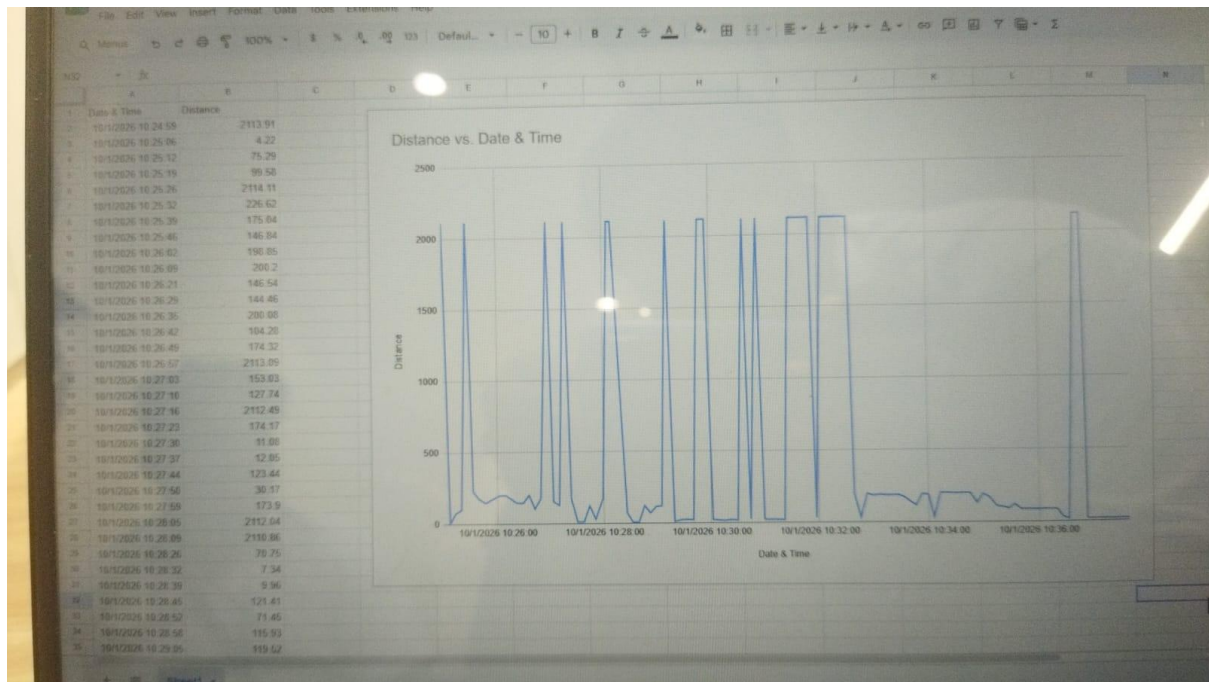
```
GPIO.cleanup()
```

App script

```
function doGet(e) {  
    var sheet = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();  
    var distance = e.parameter.distance;  
    var status;  
    if (distance < 20) {  
        status = "OBJECT TOO CLOSE";  
    } else {  
        status = "NORMAL";  
    }  
    sheet.appendRow([  
        new Date(),  
        distance,  
        status  
    ]);  
    return ContentService.createTextOutput("Data saved successfully");  
}
```

OUTPUT:





**Sri Manakula Vinayagar Engineering
College**

(An Autonomous Institution)

Department of ECE

ULC (5)	_____
ILS (5)	_____
TM (5)	_____
RAO (5)	_____
Viva (5)	_____
Total (25)	_____

RESULT:

The HC-SR04 ultrasonic sensor was successfully interfaced with the Raspberry Pi to measure object distance. The measured distance was transmitted to ThingsBoard Cloud using MQTT and stored as distance_cm telemetry. A real-time dashboard was created to visualize distance variation, and a threshold-based alarm was configured to indicate when the distance exceeded the predefined limit.

OUTCOME:

Ex.no:06	To implement IoT-based automation to control actuators using a mobile app (Blynk app).
Date:	

AIM:

To implement an IoT-based automation system using Raspberry Pi and Blynk IoT Web Dashboard to remotely control an actuator through the Internet.

SOFTWARE REQUIRED:

S. No	Device Name	Specification	Quantity
1	Raspberry Pi OS	Latest Version	1
2	Python 3	Python 3.x	1
3	Thonny IDE	-	1
4	Blynk IoT	Latest Version	1
5	Web Browser	-	1

HARDWARE REQUIRED:

S. No	Device Name	Specification	Quantity
1	Raspberry Pi	Raspberry Pi 3/4 Model B	1
2	LED	-	1
3	Resistor	200 Ω	1
4	Breadboard	-	1
5	Jumper Wires	Male-Female	As required

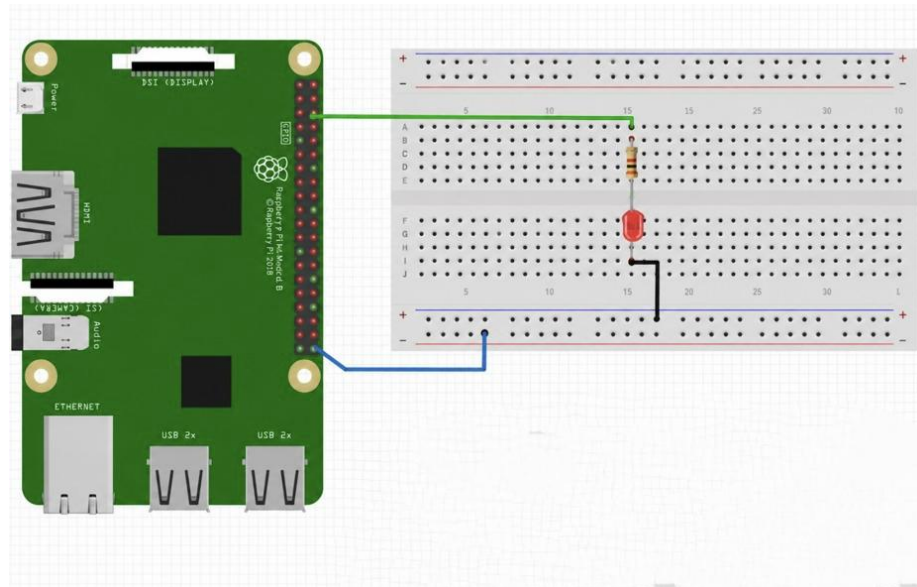
THEORY:

Blynk IoT is a cloud-based Internet of Things platform that enables electronic devices and embedded systems to be monitored and controlled remotely through the Internet. It provides a user-friendly graphical interface for developing IoT applications without requiring a dedicated web application. Blynk consists of the Blynk Console/Web Dashboard, Blynk Cloud, and the IoT device. The Web Dashboard provides widgets such as switches, buttons, gauges, labels, and charts that can be configured to interact with the connected hardware.

In this experiment, the Raspberry Pi acts as the IoT edge device and an LED is used as the actuator. The Raspberry Pi is connected to the Internet and communicates with the Blynk Cloud. A Switch widget is configured in the Blynk Web Dashboard to control the LED. The switch is associated with a Virtual Pin V0, which is used as a logical communication channel between the dashboard and the Raspberry Pi.

When the user changes the switch state in the Blynk Web Dashboard, the corresponding value is stored in the Blynk Cloud. The Raspberry Pi periodically communicates with the Blynk Cloud and obtains the current value of V0. The received value is then processed by the Python program running on the Raspberry Pi. If the value of V0 is 1, the Raspberry Pi sets its selected GPIO pin HIGH and the LED is switched ON. If the value of V0 is 0, the GPIO pin is set LOW and the LED is switched OFF. Thus, the state of the actuator can be changed remotely through the Internet.

CIRCUIT DIAGRAM:



Blynk Configuration:

Create a device in Blynk IoT and configure the following:

Parameter	Setting
Datastream	Virtual Pin
Virtual Pin	V0
Data Type	Integer
Minimum	0
Maximum	1
Widget	Switch
Switch Mode	Switch

Add a Switch widget to the Blynk Web Dashboard and assign it to V0.

The Blynk Web Dashboard is intended for visualizing data and controlling devices through widgets.

Obtain the Device Authentication Token from the device information page. Blynk's HTTPS API uses this token when requesting a datastream value.

PROGRAM:

```
import os
import ssl
import paho.mqtt.client as mqtt
from dotenv import load_dotenv
from gpiozero import LED
load_dotenv()
BLYNK_TOKEN = os.getenv("BLYNK_AUTH_TOKEN")
if not BLYNK_TOKEN:
    raise ValueError("BLYNK_AUTH_TOKEN not found")
# GPIO 17 = physical pin 11
led = LED(17)
BROKER = "blynk.cloud"
PORT = 8883
```

```

def on_connect(client, userdata, flags, reason_code, properties):
    print("Connected to Blynk Cloud!")
    print("Reason code:", reason_code)
    client.subscribe("downlink/#")
    print("Subscribed to Blynk downlink")
def on_message(client, userdata, msg):
    value = msg.payload.decode().strip()
    print("MESSAGE RECEIVED!")
    print("Topic:", msg.topic)
    print("Payload:", value)
    if value == "1":
        led.on()
        print("LED 17: ON")
    elif value == "0":
        led.off()
        print("LED 17: OFF")
client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2,
    client_id="raspberry_pi_blynk"
)
client.username_pw_set(
    username="device",
    password=BLYNK_TOKEN
)
client.tls_set(cert_reqs=ssl.CERT_REQUIRED)
client.on_connect = on_connect
client.on_message = on_message
led.off()

```

```
print("Connecting to Blynk Cloud...")
client.connect(BROKER, PORT, keepalive=45)
client.loop_forever()
```

Blynk's current HTTPS API supports retrieving a Virtual Pin value using the /external/api/get endpoint.

Important: Replace:

YOUR_BLYNK_AUTH_TOKEN

with the actual authentication token of your Blynk device.

PROCEDURE:

Step 1: Check Raspberry Pi Software

Open the Raspberry Pi terminal and enter:

```
python3 --version
pip3 --version
uname -a
```

Step 2: Create the Python Project

Enter the following commands one by one:

```
cd ~
mkdir blynk_led
cd blynk_led
python3 -m venv venv
source venv/bin/activate
python --version
pip --version
pwd
```

Step 3: Create a Blynk Template

Open the Blynk IoT app and create a new template.

Step 4: Create the Datastream

Create an Integer Datastream with:

Virtual Pin: V0

Data Type: Integer

Minimum: 0

Maximum: 1

Step 5: Create the Button

Add a Button/Switch widget in Blynk.

Connect it to:

V0

Set:

OFF = 0

ON = 1

Step 6: Create the Blynk Device

Create a device using the template.

Obtain the Device Auth Token.

Step 7: Install Python Packages

Activate the virtual environment:

```
source ~/blynk_led/venv/bin/activate
```

Install the required packages:

```
pip install paho-mqtt
```

```
pip install python-dotenv
```

```
pip install gpiozero
```

Step 8: Verify the Packages

Run:

```
pip show paho-mqtt
```

```
python -c 'import paho.mqtt.client as mqtt; print("Paho MQTT OK")'
```

```
python -c 'from dotenv import load_dotenv; print("dotenv OK")'
```

```
python -c 'from gpiozero import LED; print("gpiozero OK")'
```

Step 9: Create the .env File

Run:

```
nano .env
```

Enter:

```
BLYNK_AUTH_TOKEN=YOUR_BLYNK_AUTH_TOKEN
```

Replace YOUR_BLYNK_AUTH_TOKEN with your actual Blynk device token.

Step 10: Save the .env File

Press:

Ctrl + O

Then press:

Enter

Then:

Ctrl + X

Step 11: Protect the .env File

Run:

```
chmod 600 .env
```

Now the Blynk token is stored locally and protected.

Step 12: Test Blynk Without Hardware First

```
python blynk_test.py
```

Expected startup output:

Connecting to Blynk Cloud...

Connected to Blynk Cloud!

Reason code: Success

Subscribed to Blynk downlink

Now press the Blynk button. ON should produce payload 1; OFF should produce payload 0.

In the successful setup used for this project, the received topic appeared as downlink/ds/Integer V0. Keeping downlink/# during initial debugging avoids guessing the exact topic.

Step 13: Wire the LED

Raspberry PI	Component	Connection
Raspberry Pin 11	GPIO 17	220–330 Ω resistor
Resistor	LED anode (+)	LED long leg
LED Cathode(-)	GND	physical pin 6

Step 14: Test GPIO Separately

```
python -c "from gpiozero import LED; led=LED(17); led.on(); print('GPIO 17 ON')"
```

```
python -c "from gpiozero import LED; led=LED(17); led.off(); print('GPIO 17 OFF')"
```

If these tests work, the physical GPIO/LED path is working independently of Blynk.

Step 15: Full End-to-End Test

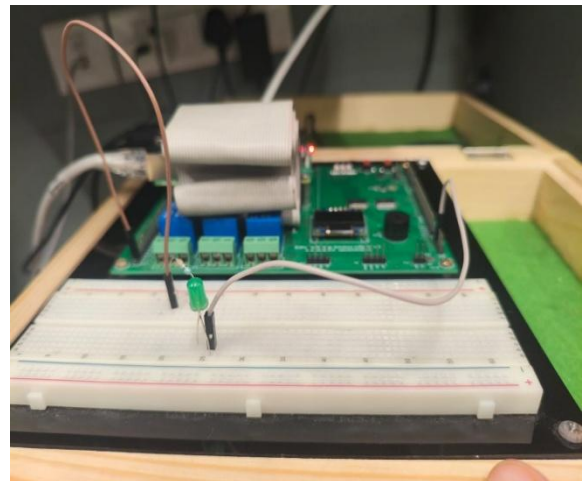
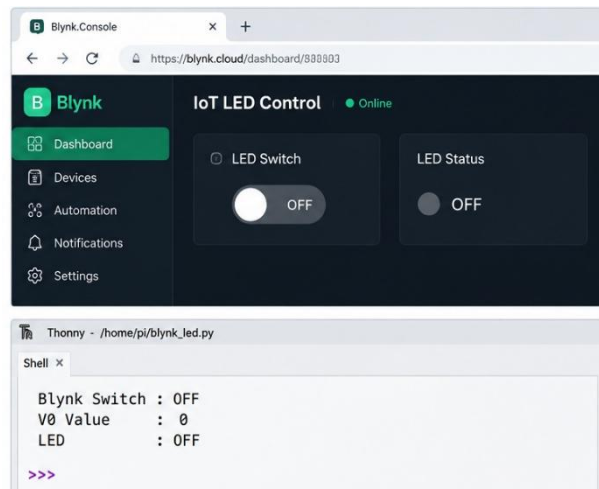
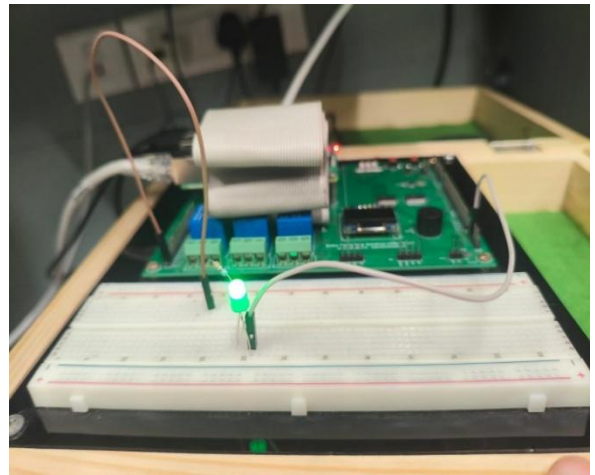
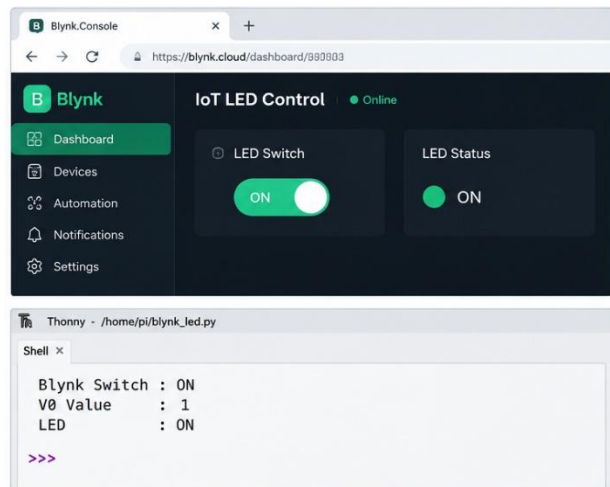
1. Activate venv
2. Run: `python blynk_test.py`
3. Confirm Blynk Cloud connection
4. Press Blynk ON
5. Confirm Payload: 1 and LED 17: ON
6. Physical LED turns ON
7. Press Blynk OFF
8. Confirm Payload: 0 and LED 17: OFF
9. Physical LED turns OFF

OBSERVATION:

S.No.	Blynk Switch	V0 Value	LED Status

OUTPUT

Blynk Switch ON



Sri Manakula Vinayagar Engineering College (An Autonomous Institution) Department of ECE	
ULC (5)	_____
ILS (5)	_____
TM (5)	_____
RAO (5)	_____
Viva (5)	_____
Total (25)	_____

RESULT:

Thus, the IoT-based automation system was successfully implemented using Raspberry Pi and Blynk Web Dashboard. The LED actuator was remotely controlled by changing the switch state through the Blynk Web Dashboard.

OUTCOME:



Ex.No.07	To develop an IoT system monitoring tool to capture and generate alerts and predict machine maintenance.
Date:	

AIM:

To familiarize with Raspberry Pi 3 by interfacing a ultrasonic sensor to practice initialization, detection of distance using Python programming.

SOFTWARE REQUIRED:

- Raspberry Pi OS
- Thonny Python IDE

HARDWARE REQUIRED:

S.No	Hardware
1.	Raspberry Pi 3 Model B
2.	Personal Computer
3.	HDMI Cable
4.	USB Keyboard
5.	USB Mouse
6.	Power Adapter

S.NO	Component/Specification	Quantity
1.	PIR Motion Sensor (HC-SR501)	1
2.	Servo Motor (SG90)	1
3.	Breadboard	1
4.	Jumper Wires	As required

THEORY:

The Raspberry Pi is a single-board computer used for controlling and interfacing with various electronic components such as sensors and actuators. It provides GPIO (General Purpose Input/Output) pins that can be configured as input or output for communicating with external devices.

An ultrasonic sensor is a distance-measuring sensor that uses high-frequency sound waves to detect the distance of an object. The commonly used HC-SR04 ultrasonic sensor consists of two main parts: a Trigger pin and an Echo pin.

The Trigger pin is used to send a short ultrasonic pulse. When the pulse reaches an object, it is reflected back towards the sensor. The Echo pin receives the reflected signal and provides a pulse whose duration corresponds to the time taken by the ultrasonic wave to travel to the object and return.

The distance is calculated using the speed of sound:

$$Distance = \frac{Time \times Speed\ of\ Sound}{2}$$

The division by 2 is required because the ultrasonic wave travels from the sensor to the object and back to the sensor.

For example, the speed of sound in air is approximately 343 m/s at room temperature. In practical applications, the distance can be calculated in centimetres using:

$$Distance(cm) = \frac{Time(\mu s) \times 0.0343}{2}$$

The Raspberry Pi reads the Echo pulse duration and calculates the distance. This value can then be displayed on the terminal or used to control an actuator such as an LED, buzzer, or motor.

WORKING PRINCIPLE:

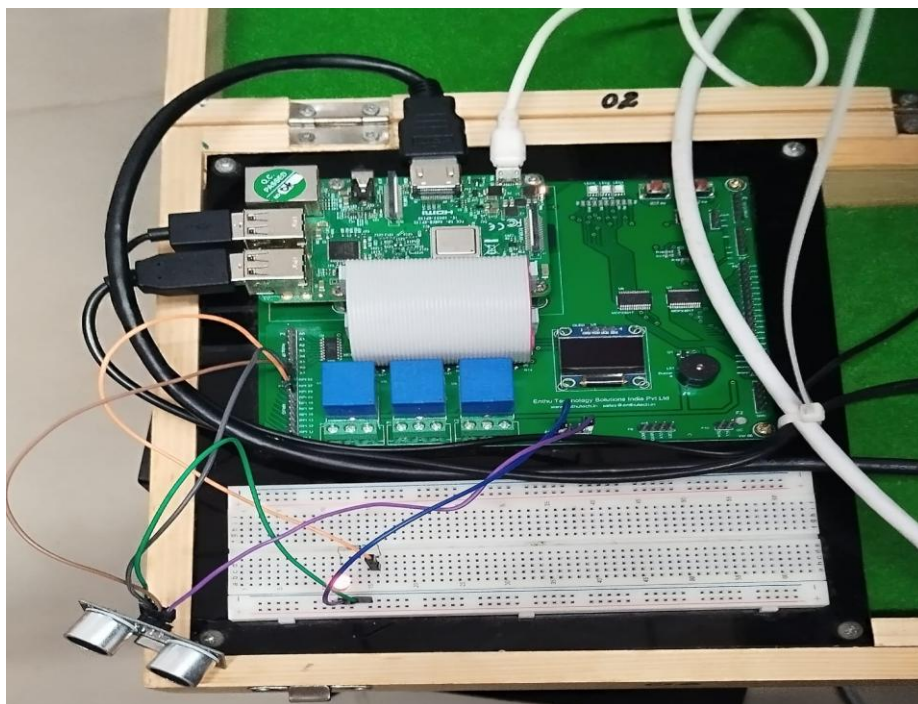
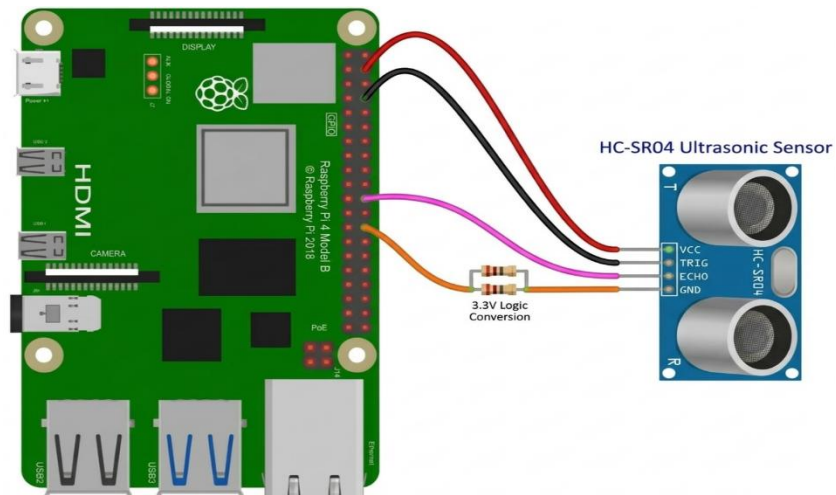
1. Raspberry Pi sends a trigger signal to the Trigger pin of the HC-SR04.
2. The ultrasonic sensor transmits an ultrasonic pulse.
3. The pulse travels through the air and hits an object.
4. The reflected pulse returns to the sensor.
5. The Echo pin generates a signal corresponding to the travel time.
6. Raspberry Pi measures this time.
7. The distance is calculated using the ultrasonic distance equation.
8. The measured distance is displayed or used for further control.

PROCEDURE:

1. Connect the Raspberry Pi to a monitor, keyboard, and mouse.
2. Power ON the Raspberry Pi.
3. Open the Thonny Python IDE.
4. Connect the HC-SR04 Ultrasonic Sensor:
 - VCC → 5V
 - GND → GND
 - TRIG → GPIO19
 - ECHO → GPIO26
5. Connect the LED:
 - Anode (+) → GPIO5 through a suitable resistor
 - Cathode (-) → GND
6. Import the required Python libraries, RPi.GPIO and time.
7. Configure GPIO19 as the TRIG output, GPIO26 as the ECHO input, and GPIO5 as the LED output.
8. Write and save the Python program.
9. Execute the program.
10. Place an object in front of the ultrasonic sensor and observe the measured distance displayed in centimetres.
11. When the object is less than 20 cm from the sensor, observe that the LED turns ON.
12. Move the object beyond 20 cm and observe that the LED turns OFF.
13. Stop the program and safely shut down the Raspberry Pi.

CIRCUIT DIAGRAM:

Ultrasonic sensor interfacing with Raspberry Pi

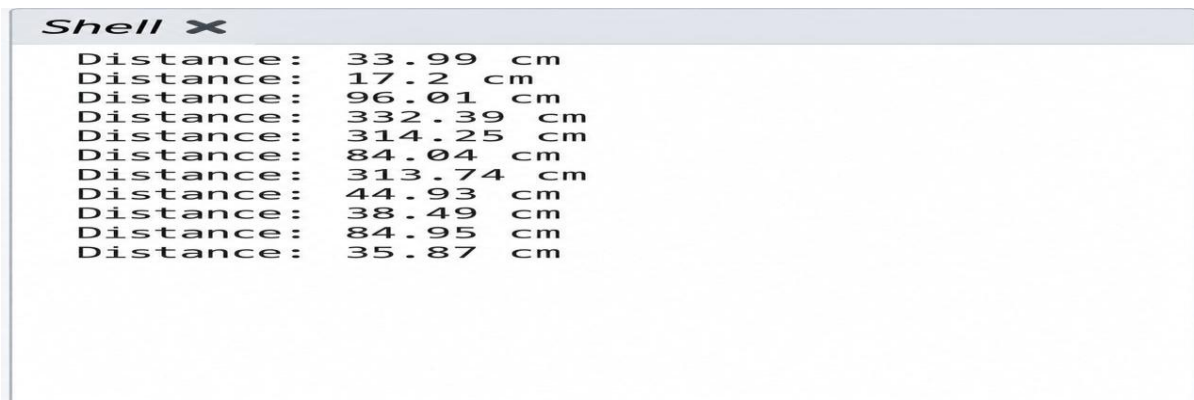


SOURCE CODE:

Ultrasonic Sensor

```
import RPi.GPIO as GPIO
import time
TRIG = 19
ECHO = 26
LED = 5
GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.setup(LED, GPIO.OUT)
try:
    while True:
        GPIO.output(TRIG, 1)
        time.sleep(0.00001)
        GPIO.output(TRIG, 0)
        while not GPIO.input(ECHO):
            start = time.time()
        while GPIO.input(ECHO):
            end = time.time()
        distance = (end - start) * 17150
        print("Distance:", round(distance, 2), "cm")
        GPIO.output(LED, distance < 20)
        time.sleep(0.2)
except KeyboardInterrupt:
    GPIO.cleanup()
```

OUTPUT:



```
Shell X
Distance: 33.99 cm
Distance: 17.2 cm
Distance: 96.01 cm
Distance: 332.39 cm
Distance: 314.25 cm
Distance: 84.04 cm
Distance: 313.74 cm
Distance: 44.93 cm
Distance: 38.49 cm
Distance: 84.95 cm
Distance: 35.87 cm
```

Sri Manakula Vinayagar Engineering College (An Autonomous Institution) Department of ECE	
ULC (5)	_____
ILS (5)	_____
TM (5)	_____
RAO (5)	_____
Viva (5)	_____
Total (25)	_____

RESULT:

The experiment was conducted to interface the HC-SR04 ultrasonic sensor with Raspberry Pi for measuring the distance of an object and controlling an LED based on the measured distance. Thus, the experiment was successfully verified.

OUTCOME:



Ex.no:08	To implement IoT security measures by understanding and identifying threats, authenticated and authorized users and communicate with encryption.
Date:	

AIM

To implement IoT security measures by understanding and identifying security threats, authenticating and authorizing users, and securely communicating IoT data using HTTPS encryption.

SOFTWARE REQUIRED:

1.	ThingsBoard Cloud
2.	Python 3
3.	Python requests library

THEORY

IoT Security

IoT security refers to protecting connected devices, networks, cloud platforms, and data from unauthorized access and malicious activities. IoT systems continuously exchange data through the Internet, so security mechanisms are required to protect sensitive information.

The major security mechanisms implemented in this experiment are:

1. Authentication

Authentication is the process of verifying the identity of a user.

In this experiment, two users are defined:

- Admin – Username: admin
- User – Username: user

The entered username and password are compared with the predefined credentials. If they match, authentication is successful.

2. Authorization

Authorization determines what an authenticated user is allowed to do.

In this experiment:

- An Admin is allowed to perform any operation.
- A normal User is allowed to perform only the read operation.
- Other operations are rejected for the normal user.

Thus, authentication verifies who the user is, while authorization determines what the user can do.

3. Encryption / Secure Communication

IoT data transmitted over the Internet should be protected against unauthorized interception.

The program communicates with the ThingSpeak API using:

HTTPS

HTTPS provides encrypted communication between the Python program and the cloud server.

4. Threats in IoT

Common IoT security threats include:

- Unauthorized access
- Password attacks
- Data interception
- Data modification
- Impersonation
- Unauthorized operations
- Insecure communication

Authentication, authorization and encrypted communication help reduce these security risks.

5. ThingSpeak Communication

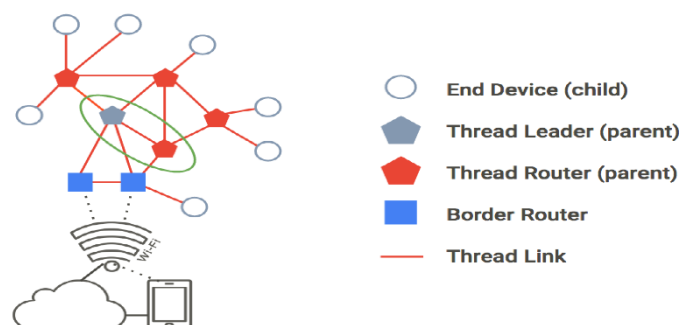
ThingSpeak is an IoT cloud platform used to collect and visualize sensor or device data.

In this experiment, Python sends the value:

field1 = 25

to the ThingSpeak update API through an HTTPS request.

Therefore, a measured distance above 50 cm produces a High Distance alarm, while a distance of 50 cm or below clears the alarm.



PROCEDURE:

1. Install Python and the requests library.
2. Enter username and password to authenticate the user.
3. Assign Admin/User role based on valid credentials.
4. Check the requested operation and authorize it according to the user role.
5. Send IoT data to ThingSpeak using HTTPS.
6. Display the authentication, authorization, and secure transmission results.

PROGRAM:

```
import requests

username = input("Username: ")
password = input("Password: ")

if username == "admin" and password == "1234":
    print("Authentication Successful")
    role = "Admin"
elif username == "user" and password == "1111":
    print("Authentication Successful")
    role = "User"
else:
    print("Authentication Failed")
    exit()

operation = input("Enter operation: ")

if role == "Admin":
    print("Authorization Successful")
    print("Operation Allowed")
elif operation == "read":
    print("Authorization Successful")
    print("Operation Allowed")
else:
    print("Authorization Failed")
```

```

print("Operation Not Allowed")

url = "https://api.thingspeak.com/update"

data = {

    "api_key": "YOUR_API_KEY",

    "field1": 25

}

response = requests.get(url, params=data)

print("Secure Data Transmission Response:", response.text)

```

OUTPUT:

```

Username: user

Password: 1111

Authentication Successful

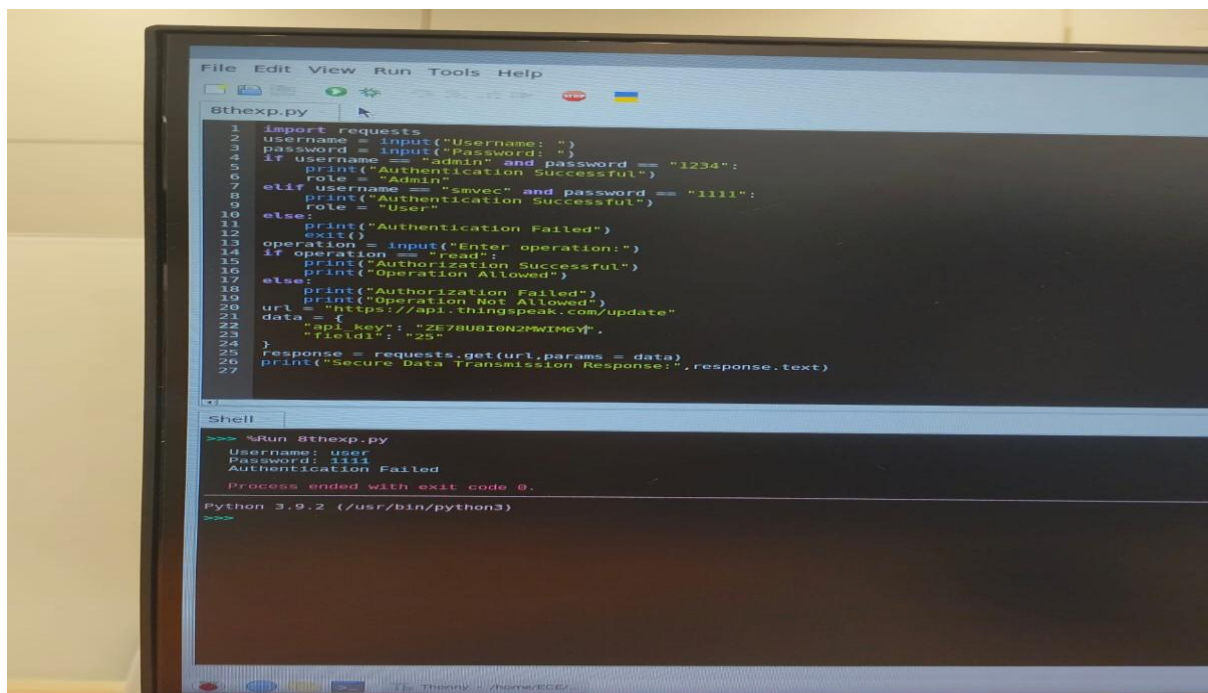
Enter operation: read

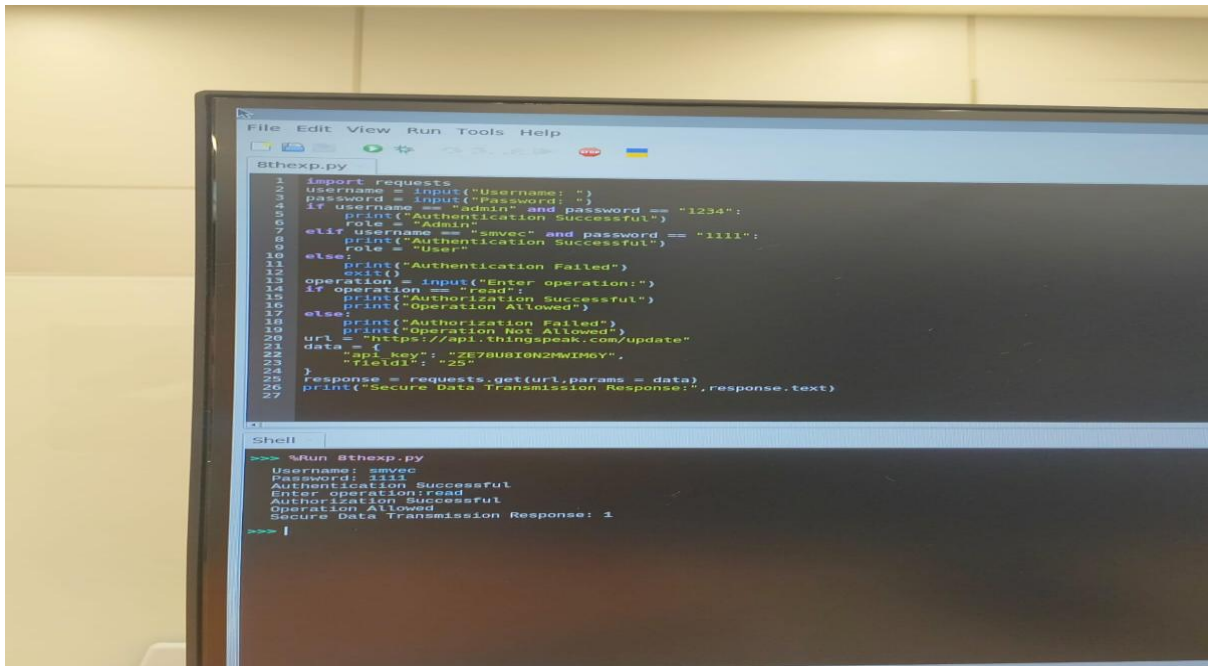
Authorization Successful

Operation Allowed

Secure Data Transmission Response: 1

```





Sri Manakula Vinayagar Engineering College

(An Autonomous Institution)

Department of ECE

ULC (5)	_____
ILS (5)	_____
TM (5)	_____
RAO (5)	_____
Viva (5)	_____
Total (25)	_____

RESULT:

Thus, the IoT security system was successfully implemented using authentication and authorization mechanisms. Different access permissions were provided to Admin and User roles, and IoT data was transmitted to ThingSpeak using HTTPS secure communication.

OUTCOME:

Ex.no:09	To build a heart monitor using pulse sensor and visualize the data to end-device via Bluetooth.
Date:	

AIM

To interface the MAX30100 pulse oximeter and heart-rate sensor with a Raspberry Pi and measure the sensor's IR and RED values using Python programming.

HARDWARE REQUIRED:

1.	Raspberry Pi
2.	MAX30100 Pulse Oximeter / Heart Rate Sensor
3.	Resistor

SOFTWARE REQUIRED:

1.	Raspberry Pi OS
2.	Python 3
3.	Thonny Python IDE
4.	MAX30100 Python library

THEORY

The MAX30100 is an optical sensor used for measuring heart rate and blood oxygen-related signals. It contains a red LED, an infrared LED, and a photodetector.

The sensor works based on the principle of photoplethysmography (PPG). When the finger is placed on the sensor, light from the LEDs passes into the finger. The amount of reflected light changes according to the blood flow caused by the heartbeat.

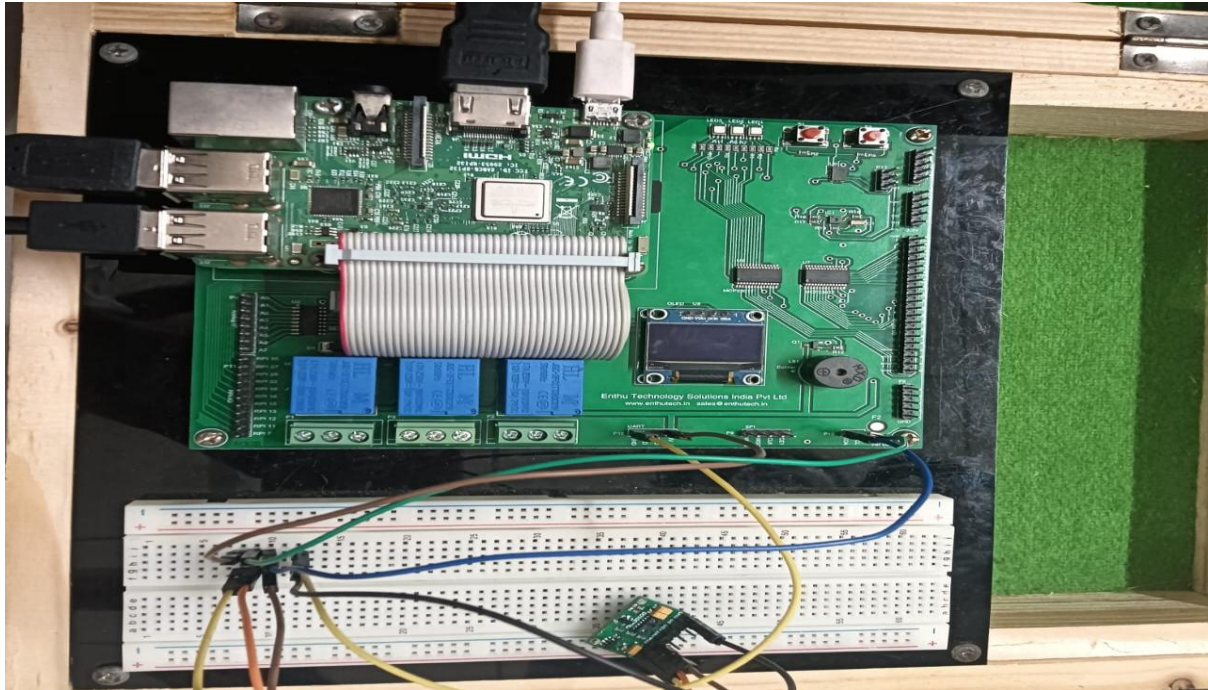
The MAX30100 provides two main readings:

- IR value – Infrared light measurement.
- RED value – Red light measurement.

The sensor communicates with the Raspberry Pi using the I²C protocol. The Raspberry Pi reads the sensor data through the I²C bus and displays the values in the Python shell.

The MAX30100 I²C address is normally 0x57.

CIRCUIT DIAGRAM



PROCEDURE:

1. Connect the MAX30100 sensor to the Raspberry Pi.
2. Connect the **VIN/VCC** pin of the sensor to the appropriate power supply.
3. Connect **GND** to Raspberry Pi GND.
4. Connect **SDA** of MAX30100 to Raspberry Pi SDA.
5. Connect **SCL** of MAX30100 to Raspberry Pi SCL.
6. Enable the **I²C interface** in Raspberry Pi configuration.
7. Open the **Thonny Python IDE**.
8. Install/import the required max30100 Python library.
9. Create a Python program to initialize the MAX30100 sensor.
10. Set the sensor operating mode to **SpO2 mode**.
11. Read the sensor using `read_sensor()`.
12. Display the IR and RED values.
13. Place a finger properly on the sensor.
14. Observe the changing sensor readings in the Thonny shell.

15. Stop the program using **Ctrl+C**.

PROGRAM

```
import max30100
import time
# Create MAX30100 object
mx30 = max30100.MAX30100()
# Set sensor to SpO2 mode
mx30.set_mode(max30100.MODE_SPO2)
try:
    while True:
        # Read sensor data
        mx30.read_sensor()
        # Display IR and RED values
        print("IR:", mx30.ir, "BPM")
        print("RED:", mx30.red)
        # Delay
        time.sleep(0.5)
except KeyboardInterrupt:
    print("Program Stopped")
```

OUTPUT:

IR: 16344 BPM

RED: 15712

IR: 16376 BPM

RED: 15736

IR: 16384 BPM

RED: 15712

Sri Manakula Vinayagar Engineering College (An Autonomous Institution) Department of ECE	
ULC (5)	_____
ILS (5)	_____
TM (5)	_____
RAO (5)	_____
Viva (5)	_____
Total (25)	_____

RESULT:

Thus, the MAX30100 sensor was interfaced with the Raspberry Pi using I²C communication, and the IR and RED values were successfully read and displayed using Python.

OUTCOME:

Ex.no:10	To build and develop an end-to-end robust IoT system/device (Applicable in real-world) encompassing data retrieval, encrypted communication, visualization, and control of the system with user-friendly application with alert generating protocols.
Date:	

AIM

To write a python code for human fall detection system using an accelerometer sensor.

APPARATUS REQUIRED:

1. ETS IoT Trainer Kit
2. ADXL335
3. Power Adaptor
4. HDMI to VGA Converter
5. Monitor

PYTHON 3 LIBRARY REQUIREMENTS:

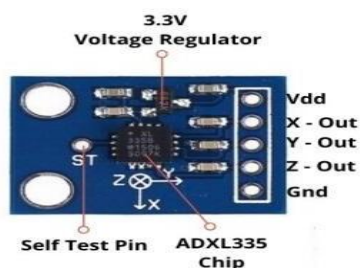
1. The user must install **Python 3 IDLE** to run the source code.
2. Here the user need not install any library for ADXL335.
3. Install library in Raspberry pi.

PROCEDURE:

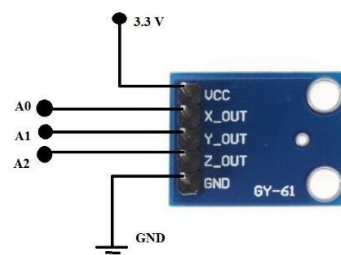
1. Switch on the ETS IoT KIT and Open Python 3 IDLE
2. Create a new file and enter the source code.
3. Connect the ADXL335 with ETS IoT KIT.
4. Save the file and run the code.
5. Observe the output of the ETS IoT Kit.

CONNECTION DIAGRAM:

ADXL335 Module



ADXL Connection WITH ETS IoT Kit



ETS IoT Kit Pin	ADXL335 Pins
A0	X_OUT
A1	Y_OUT
A2	Z_OUT

PROGRAM:

```

import time
import busio
import digitalio
import board

import adafruit_mcp3xxx.mcp3008 as MCP

from adafruit_mcp3xxx.analog_in import AnalogIn
import numpy as np

import matplotlib.pyplot as plt

spi = busio.SPI(clock=board.SCK, MISO=board.MISO, MOSI=board.MOSI) # create the
cs (chip select)

#cs = digitalio.DigitalInOut(board.D5) # GPIO
5 cs = digitalio.DigitalInOut(board.D8) # SPI0
CS0

# create the mcp object

mcp = MCP.MCP3008(spi, cs)
amp=1

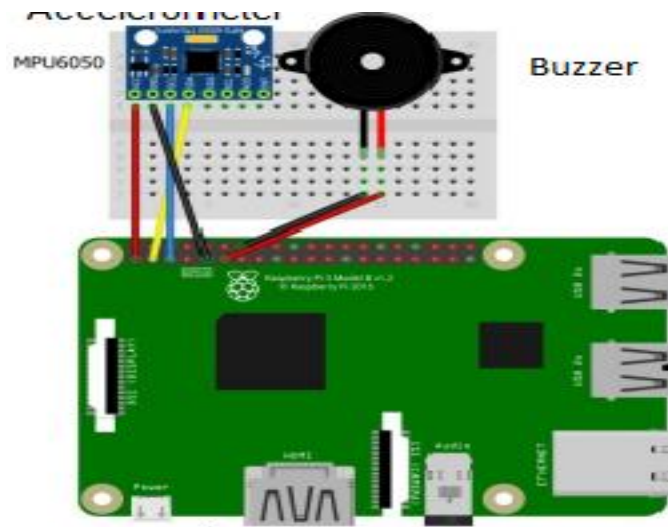
# create an analog input channel on
pin 0 chan1 = AnalogIn(mcp,
MCP.P0)

chan2 = AnalogIn(mcp,
MCP.P1)
chan3 =
AnalogIn(mcp, MCP.P2)
while
True:

    print("X_Axis: ", int(chan1.value/64))
    print("Y_Axis: ", int(chan2.value/64))
    print("Z_Axis: ", int(chan3.value/64))
    print("
_____")
    ) time.sleep(2)

```

OUTPUT:



Sri Manakula Vinayagar Engineering College (An Autonomous Institution) Department of ECE	
ULC (5)	_____
ILS (5)	_____
TM (5)	_____
RAO (5)	_____
Viva (5)	_____
Total (25)	_____

RESULT:

The ADXL335 accelerometer was successfully interfaced with the ETS IoT Kit, and the X-axis, Y-axis, and Z-axis acceleration values were successfully retrieved and displayed.

OUTCOME:
